STUDENT CLUB REGISTRATION SYSTEM

A PROJECT REPORT

Submitted by

M. ASWIN

(2116240701063)

in partial fulfilment of the End Semester Examination for the course

CB23F35 – LOW-CODE APPLICATION DEVELOPMENT USING ZOHO CREATOR

in

BACHELOR OF ENGINEERING

COMPUTER SCIENCE AND ENGINEERING



RAJALAKSHMI ENGINEERING COLLEGE

(An Autonomous Institution, Affiliated to Anna University, Chennai)

Rajalakshmi Nagar, Thandalam, Tamil Nadu – 602 105

NOVEMBER 2026

RAJALAKSHMI ENGINEERING COLLEGE

(An Autonomous Institution, Affiliated to Anna University, Chennai)

BONAFIDE CERTIFICATE

Certified that this project report titled “STUDENT CLUB REGISTRATION SYSTEM” is the bonafide work of **M. ASWIN (2116240701063)**, who carried out the individual project work on the Zoho Creator low-code platform for the course CB23F35 – Low-Code Application Development using Zoho Creator, during the academic year 2026–2027, Semester V, under my supervision.

SIGNATURE	SIGNATURE
DR.J.MANORANJINI	DR.J.JINU SOPHIA
HEAD OF THE DEPARTMENT	COURSE FACULTY
Department of Computer Science and Engineering	Associate Professor, Department of Computer Science and Engineering
Rajalakshmi Engineering College	Rajalakshmi Engineering College

Submitted for the End Semester Examination (Individual Project) held on 10/11/2026.

INTERNAL EXAMINER	EXTERNAL EXAMINER

DECLARATION

I, **M. ASWIN (2116240701063)**, hereby declare that the project report titled “STUDENT CLUB REGISTRATION SYSTEM”, submitted for the End Semester Examination of the course CB23F35 – Low-Code Application Development using Zoho Creator, is a record of original work carried out by me individually. The Zoho Creator application described in this report—including its data model, forms, reports, pages, Deluge scripts and workflows—was designed and developed by me.

I further declare that this work has not been submitted, in part or in full, for the award of any other degree or course credit. Every template, sample application, AI-assisted feature, code snippet or external resource used in this work has been duly acknowledged in the report.

SIGNATURE OF THE STUDENT

Place: Chennai
Date: 10/11/2026
M. ASWIN
2116240701063

ACKNOWLEDGEMENT

I express my sincere gratitude to our Chairman, Mr. S. Meganathan, Chairperson Dr. M. Thangam Meganathan and Vice-Chairman Mr. S. Abhay Shankar Meganathan for providing the infrastructure and facilities that made this work possible.

I am grateful to our Principal, Dr. S. N. Murugesan, Rajalakshmi Engineering College, for the constant encouragement extended to students to take up industry-relevant learning.

I thank Dr. J. Manoranjini, Head of the Department, Department of Computer Science and Engineering, for the guidance and support provided throughout the course.

I owe my sincere thanks to my course faculty, Dr. J. Jinu Sophia, Associate Professor, for the valuable guidance, reviews and feedback that shaped this application and report.

I also acknowledge Zoho Corporation and the Zoho Creator Education Programme for providing access to the platform and learning resources, and I thank my family and friends for their unwavering support.

M. ASWIN

ABSTRACT

Student clubs play an important role in campus life by encouraging technical learning, creativity, leadership and collaboration. In many institutions, club membership and event participation are managed through paper forms, spreadsheets or informal messaging. These methods can lead to duplicate registrations, incomplete student details, delayed approvals, poor visibility of club capacity and difficulty maintaining reliable participation records.

This project proposes a Student Club Registration System developed using the Zoho Creator low-code platform. The application centralises student profiles, club information, membership requests and event registrations. Students can browse clubs, submit registration requests and track their status. Club coordinators can review requests, approve or reject applications, manage club details and monitor members. Administrators maintain master data, users and access permissions. Lookup relationships connect student, club, membership and event records; validation rules prevent duplicate or incomplete submissions; Deluge workflows automate status changes and notifications. Reports and dashboard components provide visibility into membership totals, pending requests and event participation.

The system is designed to reduce manual effort, improve data consistency and make club administration transparent. Role-based access control protects student information and ensures that users can perform only the actions permitted to their responsibilities. The report presents the requirements, design, implementation plan, testing approach, deployment considerations and possible enhancements for a scalable campus registration application.

Keywords: Student Clubs; Registration System; Zoho Creator; Low-Code Development; Deluge; Workflow Automation; Role-Based Access Control

PROJECT AT A GLANCE

Table 0.1: Project profile

Item	Description
Project Title	Student Club Registration System
Application Domain	Campus Administration & Student Engagement
Student Name & Register No.	M. Aswin – 2116240701063
Platform	Zoho Creator (Education / Free Trial Programme)
Core Forms	Student Profile, Club Management, Membership Registration, Event Management, Event Registration, Approval Log, Admin Log
Reports	Student list, clubs, pending memberships, active members, event registrations
Pages / Dashboards	Student Dashboard, Club Coordinator Dashboard, Admin Dashboard
Automation	Registration validation, approval workflow, notifications, duplicate checks
Roles	Administrator, Student, Club Coordinator
Environments	Development → Stage → Production
Project Date	November 2026

TABLE OF CONTENTS

DECLARATION	iii
ACKNOWLEDGEMENT	iv
ABSTRACT	v
PROJECT AT A GLANCE	vi
CHAPTER 1 – INTRODUCTION	1
1.1 Background	1
1.2 Problem Statement	1
1.3 Objectives	1
1.4 Scope	2
1.5 Low-Code Justification	2
CHAPTER 2 – LITERATURE SURVEY AND SYSTEM STUDY	3
CHAPTER 3 – REQUIREMENT ANALYSIS	5
CHAPTER 4 – SYSTEM DESIGN	7
CHAPTER 5 – IMPLEMENTATION	10
CHAPTER 6 – TESTING AND DEBUGGING	18
CHAPTER 7 – DEPLOYMENT, SECURITY AND GOVERNANCE	20
CHAPTER 8 – RESULTS AND DISCUSSION	21
CHAPTER 9 – CONCLUSION AND FUTURE ENHANCEMENTS	23
REFERENCES	24
APPENDIX A – FEATURE COVERAGE CHECKLIST	25
APPENDIX B – COURSE OUTCOME MAPPING	27
APPENDIX C – SAMPLE DELUGE LOGIC	28
APPENDIX D – USER MANUAL	30
APPENDIX E – EVALUATION RUBRIC	31
APPENDIX F – VIVA-VOCE RECORD SHEET	33

LIST OF TABLES

Table	Title
0.1	Project profile
2.1	Manual versus low-code registration
3.1	User roles and responsibilities
3.2	Functional requirements
3.3	Non-functional requirements
3.4	Hardware and software requirements
3.5	Use case summary
4.1	Architecture layers
4.2	Entities and relationships
4.3	Membership Registration form specification
4.4	Access control matrix
5.1	Workflow scripts
5.2	Automation workflows
6.1	Test cases
6.2	Defect log
7.1	Deployment environments
8.1	Objectives versus outcomes
A.1	Feature coverage checklist
B.1	Course outcome mapping

LIST OF FIGURES

Figure	Title
4.1	System architecture
4.2	Entity relationship overview
4.3	Membership registration workflow
5.1	Application menu and modules
5.2	Student registration form
5.3	Club Management form
5.4	Membership request report
5.5	Club coordinator dashboard
5.6	Administrator dashboard

LIST OF ABBREVIATIONS

Abbreviation	Expansion
AI	Artificial Intelligence
API	Application Programming Interface
CRUD	Create, Read, Update, Delete
Deluge	Data Enriched Language for the Universal Grid Environment
KPI	Key Performance Indicator
LCAP	Low-Code Application Platform
RBAC	Role-Based Access Control
UI	User Interface
UAT	User Acceptance Testing
Zoho Creator	Low-code application development platform

CHAPTER 1

INTRODUCTION

1.1 Background

Student clubs provide a structured space for students to explore interests beyond the classroom. Technical clubs conduct workshops, hackathons and project activities, while cultural, literary and service clubs organise performances, competitions and outreach. A reliable registration process helps students discover opportunities and allows coordinators to maintain membership and participation records.

When registration is handled through paper forms, spreadsheets and messaging applications, information is spread across multiple sources. Coordinators must manually verify student details, check eligibility and communicate decisions. A centralised digital system can make this process simpler and more consistent.

1.2 Problem Statement

The existing club-registration process commonly faces the following issues:

• **Scattered information:** student and club records are stored in separate files or messages.

• **Duplicate applications:** the same student may submit more than one request for a club.

• **Slow approval:** coordinators review requests manually and applicants may not know the result.

• **Limited visibility:** club strength, vacancies and pending applications are difficult to track.

• **Event registration overhead:** attendance and participant lists are prepared separately.

• **Weak traceability:** changes to membership status may not be recorded consistently.

1.3 Objectives

• To design a central data model for students, clubs, memberships, events and event registrations.

• To provide a simple interface for browsing clubs and submitting membership requests.

• To automate duplicate checks, validations, approval decisions and notifications.

• To enable coordinators to maintain club details and manage membership requests.

• To provide reports and dashboards for membership and event participation.

• To implement role-based access and maintain an audit trail of important actions.

1.4 Scope of the Project

In scope: student profile capture, club catalogue, membership request submission, coordinator approval/rejection, event creation, event registration, status tracking, reports, dashboards and access control. **Out of scope:** fee collection, biometric attendance, integration with the university ERP and automated certificate generation; these may be considered later.

1.5 Justification for a Low-Code Approach

Zoho Creator offers visual form and report builders, relational lookup fields, workflow automation, Deluge scripting, role profiles and responsive layouts. These capabilities reduce repetitive application code and allow development effort to focus on registration rules, approval logic and usability. The platform is suitable for a small-to-medium campus workflow that needs rapid iteration.

1.6 Organisation of the Report

Chapter 2 reviews the existing system and related approaches. Chapter 3 specifies requirements and users. Chapter 4 describes architecture, data model and access design. Chapter 5 explains implementation. Chapter 6 presents testing, Chapter 7 deployment and security, Chapter 8 results and Chapter 9 concludes the work and identifies future enhancements.

CHAPTER 2

LITERATURE SURVEY AND SYSTEM STUDY

2.1 Study of the Existing System

In a manual process, students obtain club information from notices or social media, fill out a form and submit it to a coordinator. The coordinator checks eligibility and available capacity, maintains a spreadsheet and informs the student through a separate communication channel. Event registrations are often collected through another form, so membership and participation records are not linked.

2.2 Limitations of the Existing System

• Information is duplicated across spreadsheets and forms.

• Manual validation can miss incomplete or inconsistent details.

• Applicants cannot reliably track the status of their requests.

• Coordinators spend time preparing lists and sending individual updates.

• Vacancy and event-participant counts may become outdated.

• No single audit trail captures who changed a registration and when.

2.3 Manual versus Low-Code Registration

Table 2.1: Manual versus low-code registration

Criterion	Manual / Spreadsheet	Proposed Low-Code System
Data capture	Separate forms and files	Centralised validated forms
Duplicate control	Manual checking	Workflow-based duplicate check
Approval	Messages or email	Structured status and review
Status visibility	Follow-up required	Reports and dashboards
Event participants	Separate lists	Linked event registrations
Access control	File sharing	Role and record permissions
Auditability	Limited history	Action log and timestamps

2.4 Literature Survey

Research and practice in information systems consistently emphasise centralised data, consistent validation and role-aware access for administrative workflows. Low-code platforms extend these principles through visual development and configurable automation. Zoho Creator documentation describes forms, lookup relationships, reports, workflows, Deluge scripting and permission profiles that can be combined to implement a registration application. The proposed design applies these capabilities to student club administration.

2.5 Emerging Trends in Low-Code Development

• AI-assisted development and intelligent form fields can reduce repetitive data entry.

• Workflow automation can connect approvals, reminders and external services.

• Mobile-first interfaces allow students to register from personal devices

• Analytics dashboards support capacity planning and activity monitoring

• Environment-based deployment and audit controls support governance

2.6 Proposed System

The Student Club Registration System provides one application for students, club coordinators and administrators. Students maintain their profile, explore clubs and submit membership or event requests. Coordinators review requests and maintain club/event details. Administrators manage master data and access. Automated checks, notifications and dashboards make the process transparent and reduce manual effort.

CHAPTER 3

REQUIREMENT ANALYSIS

3.1 Stakeholders and User Roles

Table 3.1: User roles and responsibilities

Role	Description	Responsibilities
Administrator	System owner	Manage users, clubs, master data, permissions and audit records
Student	Applicant / member	Maintain profile, browse clubs, apply and track status
Club Coordinator	Club manager	Update club information, review requests, manage events and member lists

3.2 Functional Requirements

Table 3.2: Functional requirements

ID	Requirement	Role	Priority
FR-01	Create and update a student profile with required details	Student	High
FR-02	Browse active clubs and view description, eligibility and capacity	Student	High
FR-03	Submit a membership request for a selected club	Student	High
FR-04	Prevent duplicate active/pending registrations for the same club	System	High
FR-05	Approve or reject requests and record remarks	Coordinator	High
FR-06	Notify students when request status changes	System	Medium
FR-07	Create events and define date, venue and capacity	Coordinator	Medium
FR-08	Allow students to register for events and track status	Student	Medium
FR-09	Show club membership, pending requests and event counts	Admin, Coordinator	Medium
FR-10	Maintain audit history and role-based access	Admin	High

3.3 Non-Functional Requirements

• **Usability:** clear labels, simple navigation and responsive forms.

• **Performance:** common reports should load promptly for normal campus data volumes.

• **Security:** users must access only records and actions permitted by their role.

• **Reliability:** submitted registrations should be stored consistently and not duplicated.

• **Maintainability:** forms, workflows and scripts should be modular and documented.

• **Auditability:** approval decisions and important status changes should be traceable.

3.4 Hardware and Software Requirements

Table 3.3: Hardware and software requirements

Item	Specification
Platform	Zoho Creator (Education / Free Trial)
Scripting	Deluge
Browser	Recent Google Chrome / Microsoft Edge
Client device	Laptop / desktop / smartphone with internet access
Connectivity	Stable internet connection

3.5 Use Case Model

Table 3.5: Use case summary

Use Case	Actor	Description
UC-01 Browse clubs	Student	View active clubs, eligibility and capacity
UC-02 Apply for membership	Student	Submit a request and receive a request ID
UC-03 Review application	Coordinator	Approve/reject and enter remarks
UC-04 Register for event	Student	Select event and submit registration
UC-05 Manage club/event	Coordinator	Maintain details, capacity and schedule
UC-06 Administer system	Administrator	Manage accounts, roles and master records
UC-07 Monitor activity	Admin / Coordinator	View reports and dashboards

CHAPTER 4

SYSTEM DESIGN

4.1 System Architecture

The application follows a three-layer architecture. The presentation layer contains browser and mobile forms, reports and dashboards. The application layer contains validations, Deluge functions, approval workflows and notification logic. The data layer stores student, club, membership and event records in linked Creator forms. Authentication and role profiles control access across the layers.

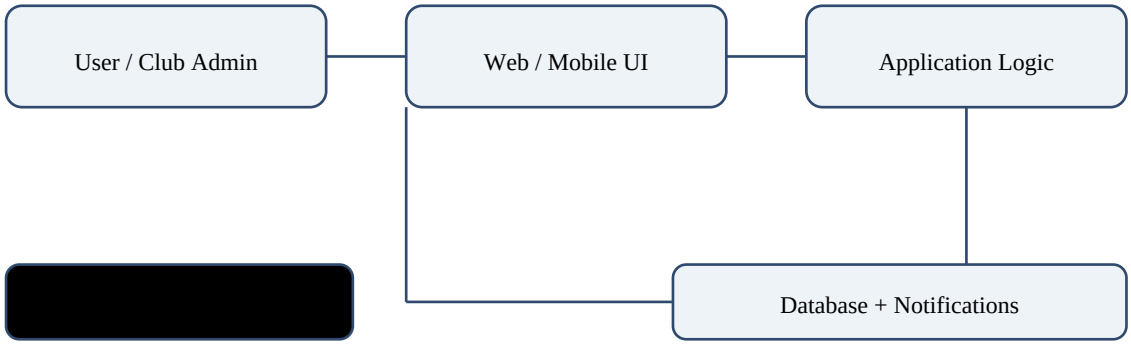


Figure 4.1: System architecture

Table 4.1: Architecture layers

Layer	Components
Presentation	Student UI, Coordinator UI, Admin dashboard, reports
Application & Logic	Form workflows, duplicate validation, approval process, Deluge scripts, notifications
Data & Storage	Creator forms, lookup relationships, registration records and audit log
Security	Authentication, role profiles and record-level criteria

4.2 Data Model

The Student Profile and Club Management forms act as master entities. Membership Registration connects a student to a club and stores application status and decision details. Event Management stores club events; Event Registration connects students to events. Admin Log records important changes. Lookup fields implement relationships and make reports consistent.

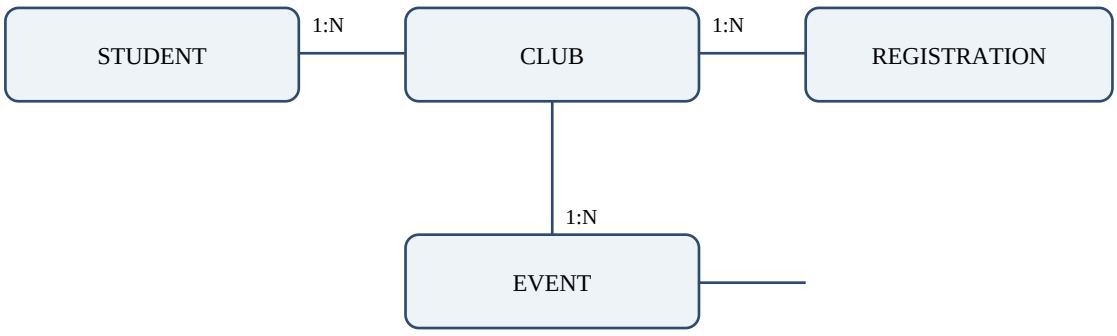


Figure 4.2: Entity relationship overview

Table 4.2: Entities and relationships

Entity	Key fields	Relationship
Student Profile	Student ID, name, register number, email, department, year	One student has many membership and event registrations
Club Management	Club ID, name, category, coordinator, capacity, status	One club has many memberships and events
Membership Registration	Request ID, student, club, date, status, remarks	Many requests belong to one student and one club
Event Management	Event ID, club, title, date, venue, capacity	One event has many event registrations
Event Registration	Event registration ID, student, event, status	Many registrations belong to one student/event
Admin Log	Action, entity, record ID, actor, timestamp	Captures important administrative changes

4.3 Form Design

Table 4.3: Membership Registration form specification

Field	Type	Validation / Remarks
Request ID	Auto Number	Generated automatically
Student	Lookup	Required; linked to Student Profile
Club	Lookup	Required; only active clubs
Application Date	Date	Default current date
Reason / Interest	Multi-line	Optional statement of interest
Status	Drop-down	Pending, Approved, Rejected, Withdrawn
Coordinator Remarks	Multi-line	Filled during review
Decision Date	Date-Time	Set when reviewed

4.4 Process and Workflow Design

A student submits a membership request. The validation workflow checks required fields, verifies that the club is active and searches for an existing pending or approved registration for the same student and club. A valid request is stored as Pending and routed to the club coordinator. The coordinator reviews the request, records a decision and remarks, and the system updates the status and sends a notification. A similar process supports event registration while checking event capacity.



Figure 4.3: Membership registration workflow

4.5 User Interface and Navigation Design

The interface is organised around role-specific navigation. Students see club browsing, their applications and event registrations. Coordinators see pending requests, club records, events and member reports. Administrators see master data, user management and system reports. Consistent labels, status indicators and concise forms reduce the effort required to complete tasks.

4.6 Access Control Design

Table 4.4: Access control matrix (V = View, A = Add, E = Edit, D = Delete)

Component	Admin	Student	Coordinator
Student Profile	AEVD	Own: V/E	V (assigned club context)
Club Management	AEVD	V	V/E (own club)
Membership Registration	AEVD	A/V (own requests)	V/E (assigned club)
Event Management	AEVD	V	AE (own club)
Event Registration	AEVD	A/V (own)	V (own events)
Admin Log	V	—	—
Dashboard	V	Student dashboard	Coordinator dashboard

CHAPTER 5

IMPLEMENTATION

5.1 Application Setup

The application is created as a custom Zoho Creator app. Modules are grouped into student-facing forms, coordinator operations and administrative reports. The home page provides a clear entry point and navigation to clubs, requests, events and dashboards. Development begins with master forms and lookup relationships before workflows are configured.

5.2 Forms and Data Capture

Student Profile captures identity and academic information such as register number, name, email, department and year. Club Management stores club name, category, description, coordinator, eligibility, capacity and active status. Membership Registration captures the student-club association and decision status. Event Management and Event Registration support event creation and participant tracking. Mandatory fields, dropdowns, lookups and validation rules maintain data consistency.

Figure 5.1: Application menu and modules

Figure 5.2: Student registration form

Figure 5.3: Club Management form

5.3 Reports

Reports provide filtered views for operational use. The Student List supports profile lookup; Active Clubs displays the catalogue; Pending Membership Requests is the coordinator's review queue; Club Members lists approved registrations; Event Registrations provides participant lists; and Admin Log Report supports traceability. Reports can be filtered by club, status, department or date where appropriate.

Figure 5.4: Membership request report

5.4 Pages and Dashboards

The Student Dashboard displays available clubs, the student's pending and approved memberships, and upcoming events. The Club Coordinator Dashboard presents pending applications, active member count, club capacity and upcoming events. The Administrator Dashboard provides totals for students, clubs, pending requests and event registrations. Charts can show membership distribution by club and request status.

Figure 5.5: Club coordinator dashboard

Figure 5.6: Administrator dashboard

5.5 Deluge Scripting

Deluge scripts implement business rules that cannot be handled by field configuration alone. The duplicate-registration check searches for an existing request with the same student and club in Pending or Approved status. If one exists, submission is stopped with a clear message. Approval scripts update decision date and status, while notification scripts inform the applicant.

Table 5.1: Workflow scripts

Script	Trigger	Purpose
DS-01 Duplicate Check	Membership form validation	Prevent duplicate pending/approved membership
DS-02 Request Initialization	Successful submission	Set Pending status and application timestamp
DS-03 Decision Update	Coordinator approval/rejection	Update status, decision date and remarks
DS-04 Student Notification	Status change	Notify student of decision
DS-05 Event Capacity Check	Event registration	Reject registration if capacity is reached
DS-06 Audit Entry	Important action	Write actor, action, record and timestamp

Code 5.1: Duplicate membership check (illustrative Deluge logic)

```
existing = Membership_Registration[Student == input.Student & Club == input.Club & (Status == "Pending" || Status == "Approved")];
if(existing.count() > 0)
{
  alert "You already have an active or pending request for this club.";
  cancel submit;
}
```

5.6 Workflows and Automation

Table 5.2: Automation workflows

Workflow	Type / Trigger	Function
Membership Validation	Form validation	Check required values and duplicates
Initialize Request	On successful submission	Set Pending status and create log entry
Membership Review	Coordinator action	Approve/reject with remarks
Status Notification	After decision	Send student notification
Event Capacity	Event registration	Check availability before accepting
Event Reminder	Scheduled	Send reminder for upcoming events

The workflow keeps the application state consistent: a request begins as Pending, can move to Approved or Rejected after review, and may be Withdrawn by the student where policy permits. Event registration similarly follows a defined status path. Notifications reduce the need for manual follow-up.

5.7 Integrations

Zoho Creator's built-in notification and email actions can be used to inform students and coordinators. If institutional messaging or calendar services are available, connections or REST APIs can be added after authentication and permission requirements are reviewed. External integrations are optional and are not required for the core registration workflow.

5.8 User Management and Role-Based Access

Administrator, Student and Club Coordinator profiles are configured with least-privilege permissions. Students can view and edit their own profile and see their own requests. Coordinators can review applications for clubs assigned to them and maintain those clubs' events. Administrators maintain the application and can inspect audit records. Record-level criteria are used where possible to enforce ownership.

5.9 AI and Analytics Features

The core release relies on structured data and native Creator analytics. Dashboard metrics include total clubs, approved members, pending requests and event participation. Future AI-assisted recommendations may suggest clubs based on student interests, but such recommendations should be optional, explainable and based only on appropriate data.

5.10 Pro-Code Extension

The initial system can be implemented using forms, reports, Deluge and workflows. A custom widget or external service may be justified later for advanced search, calendar integration or richer event visualisation; it is not necessary for the basic registration lifecycle.

5.11 Mobile and Portal Access

Responsive layouts allow students and coordinators to use the application from desktop and mobile browsers. A student portal can expose only the required student actions, while coordinator and administrator screens remain restricted to authorised users.

CHAPTER 6

TESTING AND DEBUGGING

6.1 Testing Strategy

Testing is planned at four levels: unit checks for field validation and scripts; integration checks for submission, approval and notification workflows; role-based checks for record visibility and permissions; and user acceptance testing with representative students and coordinators. Tests should be executed in the stage environment before production publishing.

6.2 Test Cases

Table 6.1: Test cases

ID	Test	Steps / Input	Expected Result
TC-01	Valid student profile	Enter all mandatory details	Profile saved
TC-02	Incomplete profile	Leave required field blank	Validation message; save blocked
TC-03	Membership request	Choose active club and submit	Request saved as Pending
TC-04	Duplicate request	Submit same club request again	Duplicate blocked
TC-05	Coordinator approval	Approve pending request	Status Approved; student notified
TC-06	Coordinator rejection	Reject with remarks	Status Rejected; remarks stored
TC-07	Inactive club	Attempt registration	Request prevented
TC-08	Event capacity	Register when full	Registration blocked
TC-09	Student access	Open another student's record	Access denied
TC-10	Audit record	Change request status	Actor and timestamp recorded

6.3 Debugging Deluge Scripts

The Deluge editor and execution logs can be used to inspect lookup results, counts and status values. Null checks should be added before accessing optional lookup fields. Workflow criteria must be tested for both positive and negative paths, including duplicate requests, inactive clubs and full events. Clear error messages should be presented to users without exposing internal implementation details

6.4 Defect Log

Table 6.2: Defect log

ID	Defect	Cause	Resolution
D-01	Duplicate request accepted	Validation omitted status filter	Add lookup count check before submission
D-02	Coordinator sees unrelated requests	Record criteria too broad	Restrict by assigned club
D-03	Event capacity exceeded	Concurrent submissions	Recheck capacity during successful submission
D-04	Notification missing	Email field empty / workflow condition	Validate recipient and workflow trigger

6.5 User Acceptance Testing

Representative users should verify that club discovery, request submission, coordinator review, event registration and status tracking are understandable and reliable. Acceptance is based on successful completion of the primary use cases, accurate dashboard values and confirmation that users cannot access unauthorised records.

CHAPTER 7

DEPLOYMENT, SECURITY AND GOVERNANCE

7.1 Environments and Deployment Strategy

Table 7.1: Deployment environments

Environment	Purpose
Development	Create forms, reports, scripts and workflows
Stage	Integration testing, role checks and user acceptance
Production	Published application for authorised campus users

Changes are developed and tested before publishing to production. Master data and user permissions are reviewed before release. A release checklist should confirm form validation, workflow triggers, dashboard calculations, notification recipients and access criteria.

7.2 Performance Optimisation

Reports should display only the columns needed for each task and use filters to avoid unnecessarily large result sets. Dashboard components should use aggregate counts rather than loading every record. Duplicate-check queries should use indexed or selective fields where available. Performance should be measured with realistic data volumes before wider deployment

7.3 Security Features

• Role-based permission profiles for administrators, students and coordinators.

• Record-level criteria to limit student and coordinator visibility.

• Authenticated access over HTTPS and platform-managed storage.

• Validation of inputs and controlled status transitions.

• Audit logging for approvals and important administrative changes.

• Periodic review of accounts, roles and inactive users.

7.4 Governance and Administration

The administrator maintains club master records, coordinator assignments, user roles and application settings. Audit entries should record the action, record identifier, actor and timestamp. Retention and access policies should follow institutional requirements, particularly for student contact information. Production changes should be documented and tested before release.

CHAPTER 8

RESULTS AND DISCUSSION

8.1 Application Output

The proposed application produces a central club catalogue, student profile records, membership requests with clear status, coordinator review queues and event participant lists. The dashboard offers a consolidated view of club activity. The system design supports traceability through decision remarks and action logs. Actual deployment metrics should be populated after the application is implemented and tested with live records

8.2 Achievement of Objectives

Table 8.1: Objectives versus outcomes

Objective	Expected outcome
Central data model	Linked student, club, membership and event records
Simple registration	Student-facing club browsing and request form
Automated review	Pending queue, decision status and notification
Reporting	Membership and event dashboards
Secure access	Role profiles and record-level restrictions
Traceability	Audit entries for key status changes

8.3 Comparison with Existing System

Table 8.2: Existing versus proposed system

Aspect	Existing process	Proposed system
Data storage	Separate sheets/forms	Linked Creator forms
Application	Manual follow-up	Structured request workflow
Duplicate handling	Manual check	Automated validation
Status	Messages / calls	Status field and dashboard
Event list	Independent spreadsheet	Linked participant records
Access	Shared files	Role-based access
History	Limited	Audit log

8.4 Limitations of the Low-Code Approach

• Platform dependency and edition limits may affect scale and feature availability.

• Highly customised UI behaviour may require widgets or external code.

• Complex concurrent capacity checks need careful workflow design.

• Institutional integrations depend on API access and approvals.

• The accuracy of dashboards depends on clean and complete source data.

8.5 Reflection on the No-Code → Low-Code → Pro-Code Spectrum

Forms, reports and dashboard components are assembled through no-code builders. Registration rules, duplicate checks and status transitions use low-code Deluge scripting. Pro-code extensions are reserved for requirements that cannot be met efficiently through built-in components. This separation keeps the first release maintainable while allowing future expansion.

CHAPTER 9

CONCLUSION AND FUTURE ENHANCEMENTS

9.1 Conclusion

The Student Club Registration System provides a structured digital approach to campus club membership and event participation. By bringing student profiles, club records, requests, approvals and events into one application, it reduces fragmented data and repetitive coordination. Role-specific interfaces, validations, workflows and reports improve transparency and help students track their applications. The low-code approach makes the solution practical to build, test and maintain while retaining scope for future integration

9.2 Future Enhancements

• Integration with the institutional student information system for verified profiles.

• Calendar synchronisation and automated reminders for club events.

• QR-based event check-in and attendance reports.

• Certificate generation for eligible event participants.

• Interest-based club recommendations with privacy safeguards.

• Waitlists and automatic vacancy allocation for full clubs.

• Advanced analytics for participation trends and club engagement.

9.3 Learning Outcomes and Career Relevance

The project provides practical experience in relational data modelling, low-code form design, lookup relationships, Deluge scripting, workflow automation, access control, reporting, testing and deployment. These skills are relevant to application development, business process automation and solution engineering.

REFERENCES

[1] Zoho Corporation, “Zoho Creator Help Documentation,” [Online]. Available: https://www.zoho.com/creator/help/

[2] Zoho Corporation, “Deluge Script Reference Guide,” [Online]. Available: https://www.zoho.com/deluge/

[3] Zoho Corporation, “Zoho Creator – Forms, Workflows and Reports,” product documentation.

[4] W. Rosam and O. Kalnichevski, *Low-Code Development: A Practical Guide*, BPB Publications.

APPENDIX A – ZOHO CREATOR FEATURE COVERAGE CHECKLIST

Table A.1: Feature coverage checklist

Feature	Planned	Where
Application and navigation	Yes	5.1
Forms with varied field types	Yes	5.2
Lookup relationships	Yes	4.2
Validation rules	Yes	5.2 / 5.5
List reports and filters	Yes	5.3
Dashboard pages	Yes	5.4
Deluge form events	Yes	5.5
Email / in-app notifications	Yes	5.6 / 5.7
Approval workflow	Yes	5.6
Scheduled reminder	Optional	5.6
Role profiles	Yes	5.8
Record-level access	Yes	4.6 / 5.8
Testing and debugging	Yes	6
Deployment environments	Yes	7.1
Audit trail	Yes	7.4
Custom widget / pro-code	Optional	5.10

APPENDIX B – COURSE OUTCOME MAPPING

Table B.1: Mapping of course outcomes to report chapters

CO	Course outcome	Evidence
CO1	Explain low-code principles and platform architecture	Chapters 1 and 2
CO2	Develop applications using forms, reports, workflows and scripts	Chapters 4 and 5
CO3	Integrate services and apply role-based security	Chapters 4, 5 and 7
CO4	Deploy, secure and manage applications	Chapter 7
CO5	Evaluate low-code strategy and future extension	Chapters 8 and 9

APPENDIX C – SAMPLE DELUGE LOGIC

C.1 Duplicate Membership Validation (illustrative)

```
existing = Membership_Registration[Student == input.Student & Club == input.Club & (Status == "Pending" || Status == "Approved")];
if(existing.count() > 0)
{
  alert "A pending or approved registration already exists.";
  cancel submit;
}
```

C.2 Initialise Request

```
input.Status = "Pending";
input.Application_Date = zoho.currentdate;
```

C.3 Record Decision (conceptual)

```
if(input.Decision == "Approve")
{
  input.Status = "Approved";
}
else
{
  input.Status = "Rejected";
}
input.Decision_Date = zoho.currenttime;
```

C.4 Audit Entry Fields

An audit record should capture the action name, affected registration or event ID, previous status, new status, logged-in user, timestamp and optional remarks. The exact field link names must match the forms created in the application.

APPENDIX D – USER MANUAL

D.1 Accessing the Application

Open the published Student Club Registration System URL in a browser and sign in using the account assigned to the user role. The landing page displays role-specific navigation.

D.2 Instructions for Students

• Complete or update the Student Profile form.

• Browse Active Clubs and read eligibility and capacity details.

• Open Membership Registration, select a club and submit the request.

• Track Pending, Approved or Rejected status in My Registrations.

• Browse upcoming events and submit an event registration.

D.3 Instructions for Club Coordinators

• Open the Pending Membership Requests report.

• Review student details and application remarks.

• Approve or reject the request and enter coordinator remarks.

• Maintain assigned club information and create events.

• Use reports to view active members and event participants.

D.4 Instructions for Administrator

• Maintain club master records and coordinator assignments.

• Manage user accounts and permission profiles.

• Review dashboards, audit records and application settings.

D.5 Troubleshooting

• If a module is not visible, confirm the assigned role profile.

• If submission is blocked, check mandatory fields and duplicate-registration messages.

• If a notification is missing, verify the student's email and workflow condition.

• If a club is unavailable, confirm its Active status and capacity.

APPENDIX E – EVALUATION RUBRIC

(For Examiner Use Only)

S.No.	Criterion	Max. Marks	Marks Awarded
1	Problem definition and requirement analysis	10	
2	System design and data model	10	
3	Forms, reports and pages	15	
4	Deluge scripting and workflows	15	
5	Integration, RBAC and analytics	10	
6	Testing, deployment and security	10	
7	Live demonstration	15	
8	Project report	5	
9	Viva-voce	10	
	TOTAL	100	

APPENDIX F – VIVA-VOCE RECORD SHEET

(For Examiner Use Only)

Student Name	M. ASWIN	Register No.	2116240701063
Department	Computer Science and Engineering	Date of Viva	10/11/2026
Application URL		Demo Verified	Yes / No
Duration			

Table F.1: Viva-Voce Questions and Responses

Q.No.	Question / Topic	Response (E / G / S / NI)	Remarks
1			
2			
3			
4			
5			
6			
7			
8			

E – Excellent G – Good S – Satisfactory NI – Needs Improvement

Table F.2: Marks Summary

Component	Max. Marks	Marks Awarded
Project Demonstration	15	
Report	5	
Viva-Voce	10	
Total	30	